

Data Lakehouse Architecture

AICB-P2T2 · Ngày 18 · Chương 4: Hạ Tầng

Giảng viên

VinUniversity · Phase 2 · AI Infrastructure Track · Tuần 4

HÃY SUY NGHĨ...

""Đổ tất cả vào S3" — works ở 10 GB, ác mộng ở 10 TB, production outage ở 10 PB. Lakehouse = ACID + cheap storage + AI workloads. Câu trả lời cho 3 era: Traditional, ML, LLM."

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Evolution + 3 Eras (Trad / ML / LLM)
2. Delta Lake: ACID + Deletion Vectors + CDF
3. Time Travel & Data Versioning
4. Apache Iceberg: Hidden Partitioning, v3
5. Query Engines (Spark/Trino/DuckDB)
6. Storage Optimization & Anti-Patterns
7. Format Internals & Performance Tuning
8. Lakehouse cho AI/ML Workloads
9. Streaming & CDC Ingestion
10. Industrial Deep Dive (AI Thực Chiến)
11. Production Ops (Catalog, DQ, Lineage, Security, FinOps)
12. Demo + Lab repo (notebooks 01–04)

Mục tiêu bài học

Sau buổi học này, bạn sẽ:

1. Hiểu evolution storage qua 3 era: **Traditional** → **ML** → **LLM**
2. Triển khai Delta/Iceberg: ACID, time travel, deletion vectors, CDF
3. So sánh Delta vs Iceberg vs Hudi → chọn theo workload (append / mutate / multimodal)
4. Thiết kế medallion Bronze/Silver/Gold cho LLM observability + RAG corpus
5. Áp dụng production ops: catalog, data contracts, lineage, FinOps

> Evolution + 3 Eras → Delta + Iceberg → Storage & Performance → AI/LLM Workloads → Industrial Cases → Production Ops → Demo

Deliverable Cuối Ngày

> Lakehouse Bronze/Silver/Gold + Delta ACID + time travel + benchmark — 1-to-1 với notebooks trong lab repo.

- **NB1** — Delta Lake table với schema enforcement + transaction log
- **NB2** — OPTIMIZE + Z-ORDER benchmark: query time trước/sau (chứng minh small-file problem)
- **NB3** — Time travel: `restoreToVersion` + MERGE upsert
- **NB4** — Medallion pipeline Bronze → Silver → Gold cho LLM observability hoặc RAG corpus

Lab repo: `github.com/VinUni-AI20k/Day18-AIInfrastructure-Lakehouse-Lab`

Evolution of Data Platforms



Key Innovation: metadata layer biến S3 object storage thành transactional store.
Enablers: Open table formats (Delta/Iceberg/Hudi) + cheap object storage (S3) + query engines (Spark/DuckDB/Trino).

3 Eras of Software: Workload Drives Storage

Aspect	Traditional (1990s–2010)	ML Era ($\sim$2012–2022)	LLM Era (2022+)
Workload	OLTP txn, BI reports	Feature eng, batch train/infer	Pretraining (T tokens), fine-tune, RAG, eval
Data shape	Tabular, 3NF normalized	Tabular + semi-structured (JSON)	Text + multimodal + embeddings
Volume	GB–TB	TB–PB	PB+ raw, 10^{12}+ tokens, billions vectors
Latency	ms (txn), hours (BI)	Min–hours (batch ML)	Hours (train) + sub-100 ms (RAG)
Schema	Schema-on-write (rigid)	Schema-on-read or hybrid	Hybrid + lineage + contracts + provenance
Compliance	SOX, PCI, GDPR	$+$ bias audit, fairness	$+$ training data provenance, copyright, hallucination
Failure mode	Schema drift, deadlock	Data drift, train/serve skew	Dedup leak, license violation, hallucination

Pattern: mỗi era thêm *một class data mới* mà era trước không có (semi-structured $\rightarrow$ multimodal+embeddings).

Storage stack **mở rộng**, không thay thế.

Storage Tech × Era Fit + LLM Demands

Tech	Traditional	ML Era	LLM Era
OLTP DB (Postgres)	**Primary system**	App backend; CDC source for ML	App backend; CDC; user feedback
Warehouse (Snowflake)	BI reporting	Feature aggregation + serving	BI dashboards on LLM telemetry
Data Lake (S3 + Hive)	—	Raw events, training datasets	Raw crawls, multimodal blobs
Lakehouse (Delta/Iceberg)	—	Unified BI + ML feature store	**Training corpus + RAG + eval + prompt logs**
Vector DB (Qdrant/Milvus)	—	Recommendation (rare)	**Online RAG** (sub-100 ms)
Multimodal (Lance)	—	—	**Video/image/audio + embeddings**

LLM-era demands trên Lakehouse:

- **Trillion-token dedup** (MinHashLSH)
- **Multimodal blobs** (Lance / Iceberg)
- **Embedding versioning** (doc_v × model_v)
- **Training data provenance** (Iceberg v3)
- **License governance** (per-doc tag)
- **Eval golden sets** (Iceberg tags)
- **Prompt + trace logs** → Bronze

LLM era *thêm tầng* (vector DB, multimodal) và *ép* lakehouse = dataset-of-record cho compliance.

Delta Lake Transaction Log



ACID: Atomicity + Consistency + Isolation + Durability trên S3 · **Concurrency:** optimistic conflict detection · **Metadata layer:** JSON log biến object store → transactional table.

Delta Lake: Thao Tác Quan Trọng

- `df.write.format("delta")` — basic write
- `MERGE INTO ...` = upsert. Workshop 100K $\sim 2\times$; prod 100M+ $\rightarrow$ 10–50$\times$ vs overwrite
- Schema evolution: `mergeSchema=true`

- Compact $\rightarrow$ target 128 MB–1 GB
- 10,000 $\times$ 1 MB $\rightarrow$ 10 $\times$ 1 GB
- Z-ORDER: 3–10$\times$ typical; 100$\times$ best case

- `VACUUM table RETAIN 168 HOURS`
- Dài = nhiều cost, audit tốt
- Ngắn = ít cost, mất rollback window
- Regulated 30+ ngày; startup 7 ngày

- Successor cho Z-ORDER: incremental re-cluster
- Không cần rewrite toàn bộ table
- GA Databricks; OSS Delta 3.3+ đang mở rộng

Deletion Vectors & Change Data Feed

Vấn đề: DELETE 1 row → rewrite cả file (write amplification 1000×).

Cách hoạt động: Lưu bitmap đánh dấu rows bị xoá trong sidecar file. Reader skip rows theo bitmap. Không rewrite Parquet.

Lợi ích:

- DELETE/UPDATE/MERGE 10–100× nhanh hơn
- GDPR `DELETE FROM ... WHERE user_id=X` từ giờ → phút
- Compact-on-read; physical removal khi OPTIMIZE

Bật: `ALTER TABLE ... SET TBLPROPERTIES('delta.enableDeletionVectors'=true)`

Use case: downstream consumer cần biết *cái gì thay đổi*, không phải full snapshot.

Cách bật: `TBLPROPERTIES (delta.enableChangeDataFeed = true)`

Đọc deltas:

```
spark.read.format("delta")
.option("readChangeFeed","true")
.option("startingVersion", 5)
.table("silver.users")
```

Output columns: `_change_type` (insert / update_pre / update_post / delete) + version + timestamp.

Pattern: Bronze CDF → Silver MERGE → Gold incremental refresh = streaming-like batch.

Time Travel: API


```python
# Version-based
df = spark.read.format("delta") \
  .option("versionAsOf", 5).load(path)

# Timestamp-based (point-in-time query)
df = spark.read.format("delta") \
  .option("timestampAsOf", "2025-01-15 00:00:00") \
  .load(path)

# Restore (rollback) --- creates new version với same data as v10
DeltaTable.forPath(spark, path).restoreToVersion(10)

# Audit trail
spark.sql("DESCRIBE HISTORY delta.`{}`".format(path)).show()
```

3 cách query history: `versionAsOf` (số version), `timestampAsOf` (point-in-time), `restoreToVersion` (rollback). `DESCRIBE HISTORY` = compliance-grade audit log đi kèm built-in.

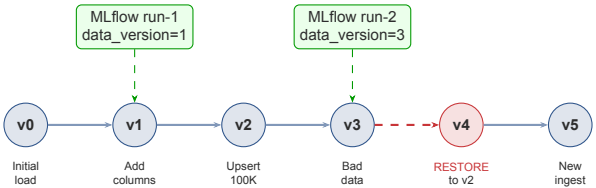
Time Travel: Use Cases & Limits

- Model reproducibility — pin training set version
- Rollback bad ingestion — instant vs 2+ giờ manual fix
- A/B test datasets — timestamp queries
- Regulatory audit — DESCRIBE HISTORY = compliance log

- Bị giới hạn bởi VACUUM retention (default 7 ngày)
- Schema-incompatible old versions có thể fail
- Khôi phục ≠ GDPR DELETE — VACUUM mới xoá vĩnh viễn

Lưu ý: Time travel + schema registry = Data Contracts.

Data Versioning & MLflow Integration



MLflow `run_id` $\leftrightarrow$ Delta table version $=$ reproducible training. Full lineage: data $\rightarrow$ features $\rightarrow$ model $\rightarrow$ deploy.

Delta Lake vs Iceberg vs Hudi

Feature	Delta Lake	Iceberg	Hudi
ACID Transactions	✓	✓	✓
Time Travel	✓	✓	✓
Deletion Vectors	✓(2.3+)	✓(v3)	✓(MOR)
Hidden Partitioning	×	✓	×
Branching/Tagging	Tag only	✓+ Nessie	×
Multi-engine native	via UniForm	✓(default)	✓
Row-level Updates	MERGE + DV	MERGE + DV	MOR (fastest)
Ecosystem (origin)	Databricks (2017)	Netflix, Apple (2018)	Uber (2016)

Iceberg metadata: `metadata.json` → `manifest list` → `manifests` → Parquet. Apache XTable convert Delta ↔ Iceberg ↔ Hudi.

Iceberg Hidden Partitioning (Game Changer)

```
-- Need extra partition column
CREATE TABLE events (
ts TIMESTAMP,
ts_day DATE,  -- duplicate!
user_id BIGINT)
PARTITIONED BY (ts_day);
-- User MUST filter ts_day
SELECT * FROM events
WHERE ts_day = '2026-04-01';
-- Forgot ts_day = full scan!
SELECT * FROM events
WHERE ts > '2026-04-01';  -- BAD
```

Vấn đề: `ts_day` duplicate dữ liệu, user dễ quên filter → full scan.

```
-- Partition is a transformation
CREATE TABLE events (
ts TIMESTAMP,
user_id BIGINT)
PARTITIONED BY (days(ts));
-- User filters natural column
SELECT * FROM events
WHERE ts > '2026-04-01';
-- Iceberg AUTO-prunes!
```

Transforms: `years / months / days / hours(ts), bucket(N, col), truncate(N, col).`

Partition Evolution: đổi `days(ts)` → `hours(ts)` không cần rewrite data.

Tại sao quan trọng: đa số performance regression trong production lakehouse là vì user *quên partition column*. Hidden partitioning loại bỏ cả class bugs này. **LinkedIn** cite đây là lý do chính migrate Hive → Iceberg.

Khi Nào Chọn Iceberg vs Delta Lake?

- Databricks ecosystem heavy
- Spark-first workloads
- Zero friction với Databricks Runtime
- Team quen thuộc Delta API

- Multi-engine: cùng 1 table query từ Spark, Trino, Flink, Snowflake
- Vendor neutrality + partition evolution
- REST Catalog: Polaris (Snowflake OSS), Nessie (git-like branching cho data!)
- GDPR row-level deletes

Adoption 2026: Netflix · Apple · LinkedIn · Adobe at scale. AWS Athena/EMR default. Snowflake native (Polaris). Databricks Iceberg v3 GA Apr 2026. Iceberg = de facto open standard.

Query Engines cho Lakehouse

Engine	Sweet Spot	Scale	Format Support
Spark SQL	ETL, batch ML pipelines	TB–PB	Delta, Iceberg, Hudi (native)
Trino	Federated BI, ad-hoc SQL	GB–PB	Iceberg (native), Delta (connector)
DuckDB	Single-node analytics, dev	MB–100GB	Parquet/Delta/Iceberg via extensions
Photon	Databricks-only fast SQL	TB–PB	Delta, Iceberg
Athena	Serverless ad-hoc on S3	GB–TB	Iceberg native, Delta read-only

- $<$100 GB, 1 dev: **DuckDB** — zero infra
- ETL Spark-native: **Spark SQL**
- BI multi-source: **Trino**
- AWS-only serverless: **Athena**

Đừng chạy Spark cluster cho 5 GB query — DuckDB nhanh hơn, rẻ gần như 0. Chỉ scale-up khi data $>$ 1 node RAM.

Columnar vs Row Storage

Row-oriented (Avro/JSON)

Row 1	id	name	age	city
Row 2	id	name	age	city
Row 3	id	name	age	city

Columnar (Parquet/ORC)

id	name	age	city
v1	v1	v1	v1
v2	v2	v2	v2
v3	v3	v3	v3

Read only age!

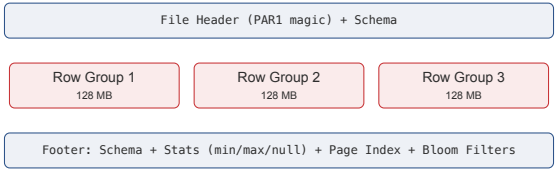
SELECT 5/100 columns → đọc ∼5% data (Parquet) vs 100% (JSON). Compression: Snappy (default, fastest), ZSTD (3× smaller, hơi chậm), GZIP (chậm nhất).

Partition Strategy cho AI Workloads

- Target: 100 MB – 1 GB mỗi partition file
- Partition by **low-cardinality**: date, region
- **Không** partition theo `user_id` (high-card)
- Over-partition → small files → slow
- Under-partition → large scans → waste IO

- **Partition**: physical directory separation
- **Z-ORDER**: co-locate within files
- Kết hợp: partition by date + Z-ORDER by `user_id`
- DuckDB: query Parquet trực tiếp từ S3
- Zero infra, ấn tượng cho $<$100 GB

Parquet Internals: Tại Sao Nhanh?



Reader strategy (4 levels of skip): (1) đọc **footer trước** ($\sim$KB) $\rightarrow$ skip Row Groups bằng min/max stats $\rightarrow$ (2) đọc **Page Index** $\rightarrow$ skip pages $\rightarrow$ (3) dùng **Bloom Filter** skip values chắc chắn không có $\rightarrow$ (4) đọc **chỉ columns trong SELECT**.

Compression: Snappy (default, fastest) · ZSTD (3$\times$ smaller, hơi chậm) · GZIP (chậm nhất). *Quy tắc:* Bronze=Snappy (fast write); Gold=ZSTD (slow write OK, fast read).

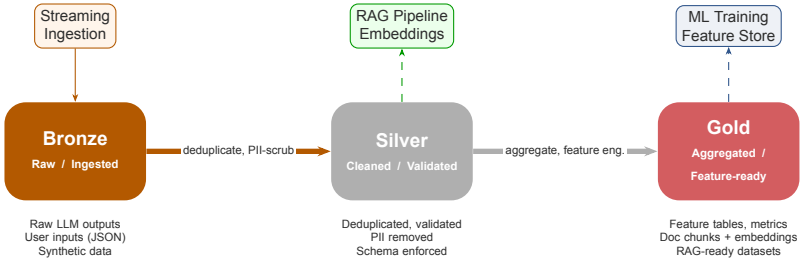
Practical impact: `WHERE user_id=42` trên 1 TB Parquet với bloom filter $\rightarrow$ đọc $\sim$50 MB. Cùng query CSV/JSON $\rightarrow$ full 1 TB. **20,000$\times$ I/O reduction**.

Schema Evolution Playbook

Change	Safe?	How	Notes
Add column (nullable)	✓	ADD COLUMN; `mergeSchema=true`	Existing rows = NULL
Add column (with default)	✓	Iceberg v3 native	Delta: backfill MERGE
Rename column	∼	Iceberg: RENAME (field ID)	Delta: cần column mapping
Drop column	∼	DROP COLUMN	OPTIMIZE physical remove
Type widen (int → bigint)	✓	ALTER COLUMN TYPE	Compatible cast
Type narrow (double → int)	×	— không cho phép	DATA LOSS
Change partition column	∼	Iceberg partition evolution	Delta: rewrite
Move column position	✓	ALTER COLUMN ... AFTER	Cosmetic, an toàn

Pattern an toàn: (1) Deploy reader code biết column mới NULL-able → (2) Add column → (3) Backfill → (4) Update writer. Đảo thứ tự = production outage.

Medallion Architecture cho AI



Medallion: Schema Cụ Thể (LLM Observability)

Layer	Bronze (raw)	Silver (clean)	Gold (analytics)
Schema	`request_id, ts, raw_json`	`request_id, ts, model, prompt_tokens, completion_tokens, latency_ms, user_id, status`	`date, model, p50/p95_latency, total_tokens, cost_usd, error_rate`
Cardinality	1 row per LLM call	1 row per call (validated)	1 row per (date, model)
Partition	`ingest_date`	`date`	`date` (Z-ORDER `model`)
Retention	30 ngày	1 năm	5 năm
Consumer	Replay / debug	Feature store, RAG corpus	Dashboards, alerts, FinOps

Quy tắc: Bronze append-only (immutable audit), Silver upsert (MERGE), Gold rebuild-from-Silver (idempotent).

Schema rõ ràng mỗi layer = data contract giữa teams.

Lakehouse + AI: Production Patterns

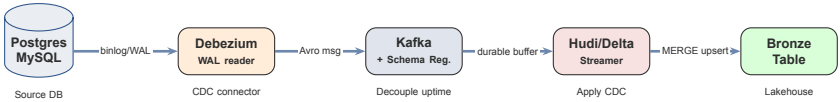
- Raw → processed → model → deploy version chain
- RAG: doc chunks + embeddings lưu Gold
- Embedding gắn doc version = fully traceable
- Pin Delta version trong MLflow run cho reproducibility

- S3 Intelligent-Tiering cho cold Bronze data
- Glacier sau 90 ngày → −60% storage cost
- Unity/Polaris: fine-grained access + audit

- Inference req/resp → Bronze (raw JSON, 30d)
- Dedup + parse tokens/latency → Silver
- Aggregate (date, model) metrics → Gold

- LLM generate → Bronze (output + prompt)
- Quality filter + MinHash dedup → Silver
- Curated set + license tags → Gold

CDC Pattern: Postgres → Lakehouse



3 failure modes phổ biến:
(1) Kafka full → Postgres WAL fills → DB outage. *Fix:* alert on Debezium lag + Kafka retention dài.
(2) Source schema change breaks pipeline. *Fix:* Schema Registry + `mergeSchema=true` ở sink.
(3) Out-of-order events tạo bad updates. *Fix:* `MERGE ... WHEN MATCHED AND src.ts > tgt.ts`.

Vietnam context: chuẩn cho fintech VN (MoMo, VNPay, Cake) — Postgres OLTP → Iceberg/Delta analytics.

End-to-end freshness 1–5 phút, không impact OLTP.

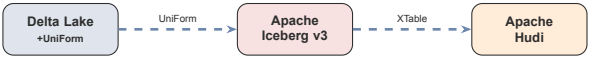
Case Studies: Lakehouse ở Quy Mô Production

Công ty	Format & Scale	Số liệu công khai
Uber	Hudi · 350 PB	6 T rows/day · 19,500 datasets · freshness 24h → 1h
Netflix	Iceberg + Lance	Atlas query planning 9.6 min → 42 sec · Media Data Lake multimodal
Apple	Iceberg	"Foundation cho lakehouse on all divisions" (MB → PB)
LinkedIn	Iceberg	Migrate Hive → Iceberg vì hidden partitioning + query planning
Shopify	Iceberg + Trino	Open lakehouse, multi-engine BI + ML

- Append-mostly (logs, events) → Delta/Iceberg
- Mutation-heavy (orders, sessions) → Hudi
- Multimodal (video, embeddings) → Lance + Iceberg

MoMo, Zalo, Shopee VN → TB–PB. Pattern chuẩn: Postgres → Debezium → Kafka → Iceberg/MinIO. Decree 13 → on-prem MinIO khả thi.

Format War 2024–2026: Đã Kết Thúc



Key events: Databricks acquires Tabular $1B+ (2024) · Snowflake → Iceberg native + Polaris catalog · Iceberg v3 GA on Databricks (Apr 2026): deletion vectors + row lineage + VARIANT · Result: **30% giảm DE workload**.

- AWS Glue (∼39% share) — default AWS
- Unity Catalog — Databricks OSS 2024
- Apache Polaris — top-level 2025
- Project Nessie — git-like branching
- Lakekeeper — Rust, K8s-native

REST Catalog spec = lingua franca 2026.

```
nessie tag create v1-prod
nessie branch create exp-2026
# ...train + evaluate on branch...
nessie merge → main
```

"Model X dùng data nào?" → `nessie tag list`: trả lời 1 command, thay vì Confluence rotting.

Lakehouse cho AI/LLM Workloads

RedPajama-V2: 30T tokens · MinHashLSH dedup
Dolma (Ai2): 3T tokens · two-stage Bloom filter
Pipeline: 64 CPU cores, $\sim$1.4 TB peak RAM

Lakehouse: Bronze (raw) $\rightarrow$ Silver (dedup) $\rightarrow$ Gold (train-ready). Time travel revert dedup mistakes.

- Random access $\sim$2000$\times$ faster than Parquet
- Native HNSW vector index; first-class blobs
- Built-in versioning

Netflix Media Data Lake: Parquet $\rightarrow$ Lance cho video frames.

Embedding version $=$ doc_v $\times$ model_v $\rightarrow$ pin via Iceberg/Delta version + MLflow `run_id`.

Vector DB (Qdrant/Milvus) $=$ **derived index**, rebuildable, không phải system of record.

- Iceberg/Delta $\rightarrow$ tabular features + lineage
- Lance $\rightarrow$ embeddings + multimodal blobs
- Vector DB $\rightarrow$ online ANN (sub-100 ms)

Bổ trợ, không thay thế nhau.

AI Thực Chiến VN: fine-tune corpus (Wiki-VI + ZNews) $\rightarrow$ apply RedPajama-style MinHash trước. Repo: `togethercomputer/RedPajama-Data`.

Catalog Layer: REST Standard 2026

Catalog	Origin	Killer feature	Khi nào dùng
AWS Glue	AWS, 2017	Default AWS, ∼39% share	Đã ở AWS
Unity Catalog	Databricks, OSS 2024	Fine-grained governance	Databricks-native
Apache Polaris	Snowflake, top-level 2025	Vendor-neutral REST	Multi-cloud
Project Nessie	Dremio, 2020	Git-like branching/tagging	ML versioning
Lakekeeper	OSS 2024 (Rust)	Lightweight, K8s-native	Self-hosted

Standard HTTP API cho Iceberg metadata (createTable, loadTable, commit, listNamespaces).

Lợi: engine (Spark, Trino, Flink, Snowflake, DuckDB) plug-and-play với bất kỳ catalog implement spec. 2026 default.

Catalog-level commit = atomic update nhiều tables.

Use case: Bronze + Silver + Gold update phải all-or-nothing.

Nessie native (git commit); Polaris đang implement.

Data Quality & Contracts: 3-Tool Stack

Where: Bronze ingestion

What: validate raw data

Strength: 50+ expectations

```
expect_column_values_to_be_in_se
column="status",
value_set=["ok",
"rate_limited","error"])
```

Where: Silver/Gold transforms

What: structural correctness

Strength: SQL-native

```
# schema.yml
- name: customer_id
tests:
- unique
- not_null
- relationships:
to: ref('dim_customers')
```

Where: Prod monitoring

What: continuous, anomaly

Strength: SodaCL DSL

```
checks for gold.metrics:
- row_count >= 1000
- freshness < 1h
- anomaly_score
< 0.7 for cost_usd
```

Data Contract: schema + constraints + freshness SLA + ownership; run trong CI (pre-merge) *và* runtime (per-batch). Bể contract → block pipeline.

Data Lineage: OpenLineage + Marquez



Pattern: bật OpenLineage **từ ngày 1**. Spark: `spark.openlineage.transport.type=http` + Marquez URL.

Security & Governance cho Lakehouse

- **RBAC:** role-based (admin/analyst/eng)
- **ABAC:** attribute-based (region, classification) — Unity/Polaris
- **Row-level security:** dynamic filter
- **Column masking:** hash/redact PII

- Tokenize PII at Bronze landing
- Encrypt at field level (Iceberg v3 native)
- Right-to-forget: DELETE + VACUUM sau 30d grace
- Audit log mọi PII column access

Decree 13/2023/NĐ-CP (eff. 2023-07-01):

- Personal data: basic vs sensitive
- Data residency: sensitive ở VN
- Cross-border: consent + DPI
- Right-to-forget: 72h SLA

Impact: sensitive data → on-prem MinIO + Iceberg.

- Every read/write → Bronze audit table
- Iceberg: `system.snapshots` table
- Retention 365+ ngày cho regulated

Cost & FinOps cho Lakehouse

Component (100 TB/tháng)	Snowflake	Iceberg + Trino / S3	Databricks
Storage	$4,000	$2,300 (S3)	$2,300 (S3)
Compute (BI)	$15,000	$8,000 (Trino EC2)	$11,000 (DBU)
Compute (ETL)	$5,000	$2,000 (Spark EMR)	$4,000 (DBU)
Catalog/gov	included	$500 (Polaris OSS)	included (Unity)
Total	**$24,000**	**$12,800 (–47%)**	**$17,300 (–28%)**

- Per-layer budget: Bronze → Glacier sau 30d
- Auto-OPTIMIZE nightly cron
- Tag clusters: `team, purpose, expires_at`
- Spot cho ETL, on-demand cho BI
- Top-10 query review hàng tháng

- Cloudera: TPC-DS 24s → 1.8s (13×), storage −36%
- ClickHouse vs Snowflake: ∼4× lower TCO
- Netflix Iceberg: planning 9.6min → 42s (14×)
- No OPTIMIZE: 10× *more* expensive than DW

Quy tắc: savings *chỉ* materialize nếu actively optimize. Forget OPTIMIZE → small-file tax giết economics.

Top 5 Lakehouse Anti-Patterns

#	Anti-pattern	Hậu quả & Fix
1	“Đổ tất cả vào S3” (raw JSON, no schema)	Data swamp $\rightarrow$ enforce schema từ Bronze; dùng Delta/Iceberg ngay từ đầu
2	Partition theo high-cardinality (vd. `user_id`)	Triệu partition nhỏ $\rightarrow$ partition by date/region, Z-ORDER `user_id`
3	Bỏ qua OPTIMIZE $\rightarrow$ small-file problem	10K files $\times$ 1 MB $\rightarrow$ query 10$\times$ chậm $\rightarrow$ daily OPTIMIZE cron
4	`VACUUM 0 HOURS` để “tiết kiệm storage”	Mất time travel + concurrent readers fail $\rightarrow$ giữ tối thiểu 168h (7 ngày, default)
5	Spark cluster cho query 5 GB	Lãng phí 10$\times$ chi phí $\rightarrow$ DuckDB / Athena cho $<$100 GB; chỉ scale-up khi $>$ 1 node RAM

80% lakehouse pain trong production $=$ một trong 5 anti-patterns này. Audit checklist trước khi deploy.

Demo: Delta Lake Time Travel & MERGE

- **Small-file problem** — ingest 1M rows (200 batches) → OPTIMIZE + Z-ORDER → benchmark trước/sau (target $\geq 3\times$)
- **Time travel rollback** — inject bad data → `restoreToVersion()` trong 30s vs $\sim$2h manual
- **MERGE upsert** — 100K rows ($\sim 2\times$ faster workshop; 10–50$\times$ production scale)
- **Audit trail** — `DESCRIBE HISTORY` list mọi operation

Lab repo: `VinUni-AI20k/Day18-AIInfrastructure-Lakehouse-Lab`

Notebooks 01–04 + Docker stack hoặc lightweight DuckDB + delta-rs path.

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **Lakehouse = ACID + object storage + open formats**. Foundation chung cho 3 era: Traditional, ML, LLM.
2. **Format war kết thúc**. Iceberg + Delta UniForm = de facto standard. On-disk Parquet identical, chọn theo tooling fit.
3. **Time travel + branching** (Nessie) = "git checkout" cho dataset. **OPTIMIZE + Z-ORDER + Deletion Vectors** bắt buộc cho production.
4. **LLM era cần thêm tầng**: Vector DB (RAG), Lance (multimodal), embedding versioning, training data provenance.
5. **Production ops trifecta**: Catalog + Data Contracts + Lineage. Bật từ ngày 1.

Ngày 19: Vector Store & Feature Store

"SQL trả exact match. AI cần tương tự — semantic search thay đổi mọi thứ."

- Hoàn thành Lab 18 (4 notebooks): `VinUni-AI20k/Day18-AIInfrastructure-Lakehouse-Lab`
- Đọc trước: case studies (Netflix, Uber, Apple Iceberg) + Lance multimodal docs
- Cài sẵn Docker + Qdrant image cho Ngày 19 (Vector Store + ANN)

Hỏi & Đáp

Câu hỏi về Lakehouse, Delta/Iceberg, Medallion, Catalog, Data Contracts, hay AI/LLM workloads?



Cảm ơn!

AICB-P2T2 · Ngày 18

Data Lakehouse Architecture

`lms.vinuni.edu.vn` · Lab repo + slides trên LMS